

Action 和 Plugin 机制

目录

01 工作原理

02 灵活配置

03 示例



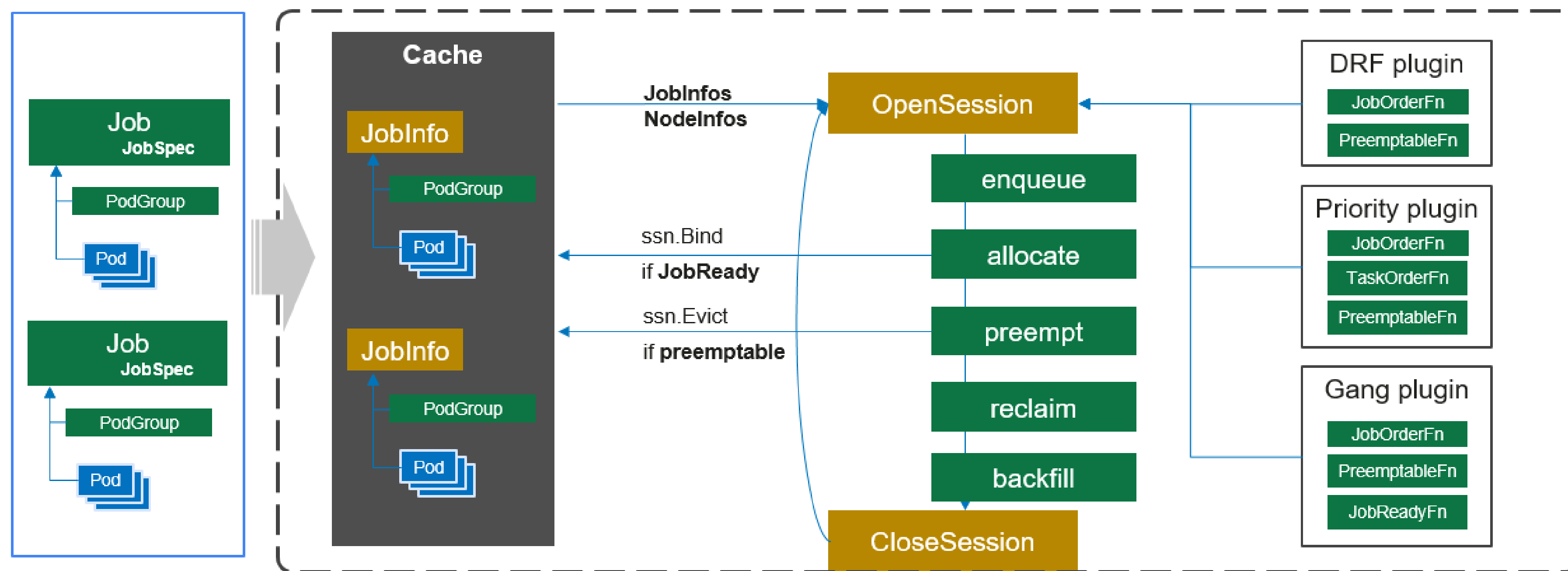
01

工作原理



调度概述

调度器分为三部分，第一层是Cache，中间部分是整个调度过程，右边是插件形式的调度算法。



Action & Plugin 接口

```
// Action is the interface of scheduler action.
type Action interface {
    // The unique name of Action.
    Name() string

    // Initialize initializes the allocator plugins.
    Initialize()

    // Execute allocates the cluster's resources into each queue.
    Execute(ssn *Session)

    // UnInitialize un-initializes the allocator plugins.
    UnInitialize()
}

// Plugin is the interface of scheduler plugin
type Plugin interface {
    // The unique name of Plugin.
    Name() string

    onSessionOpen(ssn *Session)
    onSessionClose(ssn *Session)
}
```

```
// AddJobOrderFn add job order function
func (ssn *Session) AddJobOrderFn(name string, cf api.CompareFn) {
    ssn.jobOrderFns[name] = cf
}

// AddQueueOrderFn add queue order function
func (ssn *Session) AddQueueOrderFn(name string, qf api.CompareFn) {
    ssn.queueOrderFns[name] = qf
}

// AddClusterOrderFn add queue order function
func (ssn *Session) AddClusterOrderFn(name string, qf api.CompareFn) {
    ssn.clusterOrderFns[name] = qf
}

// AddTaskOrderFn add task order function
func (ssn *Session) AddTaskOrderFn(name string, cf api.CompareFn) {
    ssn.taskOrderFns[name] = cf
}

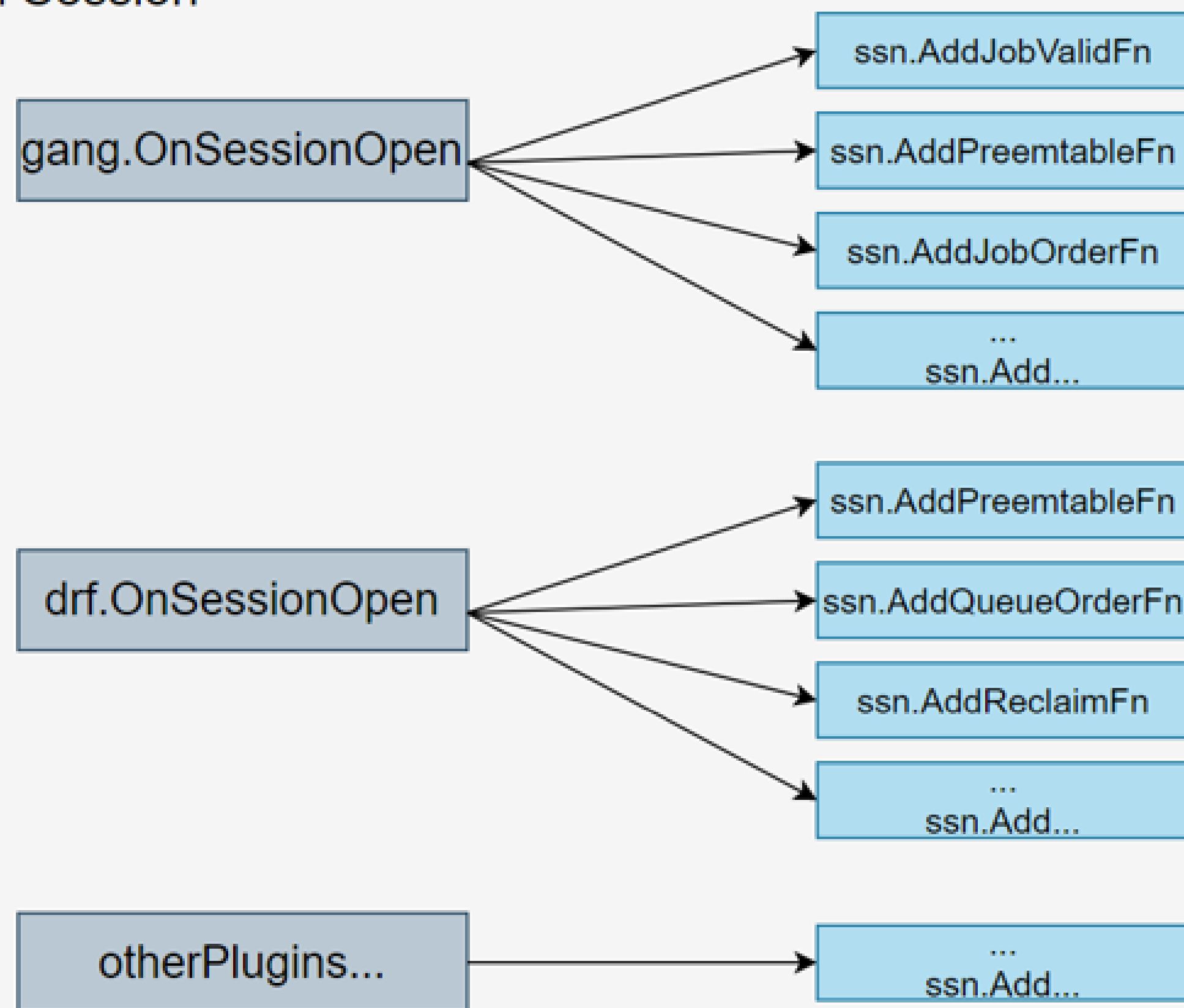
// AddNamespaceOrderFn add namespace order function
func (ssn *Session) AddNamespaceOrderFn(name string, cf api.CompareFn) {
    ssn.namespaceOrderFns[name] = cf
}

// ...
```



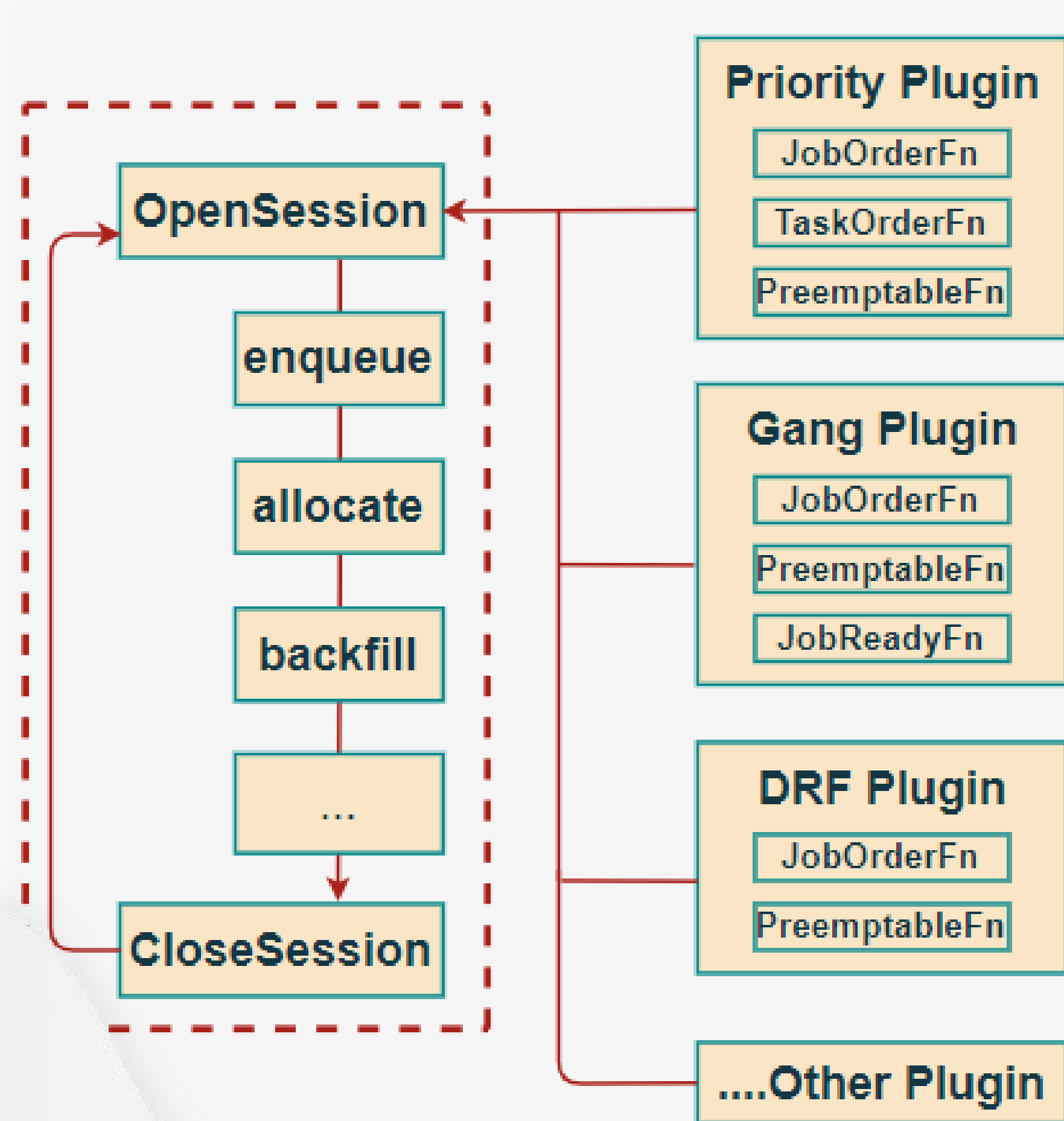
Plugin 注册

Open Session



- 调度器将为每一轮会话加载插件。
- 在开启会话时，插件会注册相应的辅助函数。

Action 执行



- 每一轮调度都会依次调用每个action(默认是 enqueue, allocate, backfill)。
- 对于每个action, 会有多个插件注册的函数被使用。
- 例如, 对于enqueue action, 使用以下函数:
 1. QueueOrderFn (plugins: Proportion, DRF)
 2. JobOrderFn (plugins: Priority, DRF, Gang, Sla, Tdm)
 3. JobEnqueueableFn (plugins: Proportion, Overcommit, Sla)

02

灵活配置

调度策略配置

- Actions
- Tiers
- Plugins
- Options

```
actions: "list_of_action_in_order"
tiers:
- plugins:
  - name: "plugin_1"
    enableJobOrder: false
  - name: "plugin_2"
- plugins:
  - name: "plugin_3"
    enableJobOrder: false
```

```
actions: "enqueue, allocate, backfill"
tiers:
- plugins:
  - name: "priority"
  - name: "gang"
  - name: "conformance"
- plugins:
  - name: "overcommit"
  - name: "drf"
  - name: "predicates"
  - name: "proportion"
  - name: "nodeorder"
  - name: "binpack"
```

配置详解

- Actions
 - 调度器将会按顺序执行列表中的action，各action之间使用逗号分隔。
- Tiers
 - 插件列表可分层配置。
 - 如果通过前序层级的插件可以得到投票结果，action将不会再调用后续层级的插件。
- Options
 - options 定义了每个插件的详细行为，例如：是否禁用该插件的抢占逻辑，如果没有特别指定，默认是不禁用。目前支持的控制的行为有preemptable, jobOrder 和 taskOrder。

Tiers 案例

```
func (ssn *Session) BatchNodeOrderFn(task *api.TaskInfo, nodes []*api.NodeInfo)
(map[string]float64, error) {
    priorityScore := make(map[string]float64, len(nodes))
    for _, tier := range ssn.Tiers {
        for _, plugin := range tier.Plugins {
            if !isEnabled(plugin.EnabledNodeOrder) {
                continue
            }
            pfn, found := ssn.batchNodeOrderFns[plugin.Name]
            if !found {
                continue
            }
            score, err := pfn(task, nodes)
            if err != nil {
                return nil, err
            }
            for nodeName, score := range score {
                priorityScore[nodeName] += score
            }
        }
    }
    return priorityScore, nil
}
```

● 两层插件都需要工作

以BatchNodeOrderFn为例，该函数是为task匹配节点打分的一个辅助函数。

BatchNodeOrderFn会调用所有注册该函数的plugin，并执行他们的实现逻辑，得到每一个plugin的打分情况，最后返回一个总分。

Tiers 案例

```
func (ssn *Session) JobEnqueueable(obj interface{}) bool {
    var hasFound bool
    for _, tier := range ssn.Tiers {
        for _, plugin := range tier.Plugins {
            if !isEnabled(plugin.EnabledJobEnqueued) {
                continue
            }
            fn, found := ssn.jobEnqueueableFns[plugin.Name]
            if !found {
                continue
            }
            res := fn(obj)
            if res < 0 {
                return false
            }
            if res > 0 {
                hasFound = true
            }
        }
        if hasFound {
            return true
        }
    }
    return true
}
```

- 第一层plugin没有得到结果，第二层plugin继续工作

以jobEnqueueable为例，该函数的功能是决定job能否进入待调度队列。

如果上层有plugin投票拒绝，那么直接返回结果false。

如果上层的plugin投票结果只有同意和弃权，那么返回结果true，不需要下层插件再进行投票表决。

只有在上层plugin全部投票弃权的情况下，才会进入下一层继续投票。

也就是说，只要上一层的plugin表决出结果，就不需要下一层plugin模型。

Plugin分层配置

- 如何确定哪些插件分组到一个层?
- 业务中配置多少层合适?

在大多数场景下，用户可以不用关心如何将plugin划分到不同层。可以在单层中配置所有plugin。只有在与驱逐相关的条件下，才需要考虑如何将plugin组织在不同的层级。例如，当启用 reclaim action 时，调度程序将尝试收集一组驱逐目标。为了减少对用户业务的影响，尽量少的驱逐目标才是合理的。可以在第一层中配置驱逐目标较少的plugin。在第二层配置驱逐目标较多的plugin。如果第一层可以挑选出目标，就不会调用注册在第二层的plugin。目前来说，一般配置两层就足够了。

待开发特性

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: volcano-scheduler-configmap
  namespace: volcano-system
data:
  volcano-scheduler.conf: |
    actions: "enqueue, allocate, backfill"
    tiers:
    - plugins:
      - name: priority
      - name: gang
      - name: conformance
    - plugins:
      - name: overcommit
      - name: drf
      - name: predicates
      - name: proportion
      - name: nodeorder
      - name: binpack
```

目前, scheduler-conf的ConfigMap为全局统一配置。

不同队列中的不同作业类型可能需要不同的调度策略。

对不同的队列使用不同的scheduler-conf。



03

示例



Actions

- 截止到2022年4月，Volcano 提供了7个内置的action，详情如下：

ID	Name	Required	Description
1	enqueue	Y	判断集群中的空闲资源是否能满足作业调度的基本需求，如果能，则将作业的podgroup设置为inqueue状态，否则保持podgroup为pending状态。
2	allocate	Y	尝试为inqueue状态的podgroup作业分配资源。
3	backfill	N	尝试为未指明pod资源申请量的作业分配资源。
4	preempt	N	识别高优先级的调度作业。尝试驱逐低优先级的Pod，并为高优先级作业分配资源。
5	reclaim	N	选择资源被其他queue占用的queue，并回收相应资源。
6	elect	N	选择满足某些条件的作业。它旨在与目标作业的资源预留配合使用。将在未来的版本中弃用。
7	reserve	N	选择一系列节点并预留资源。它旨在与目标作业的资源预留配合使用。将在未来的版本中弃用。

Plugins

- 截止到2022年4月，Volcano提供了15个内置的plugin，详情如下：

ID	Name	Arguments	Registered Functions	Description
1	binpack	* binpack.weight * binpack.cpu * binpack.memory * binpack.resources	* nodeOrderFn	尽量将Pod绑定到资源利用率高的节点上，以减少碎片化。
2	conformance	/	* preemptableFn * reclaimableFn	跳过关键Pod，而不是驱逐它们。
3	drf	/	* preemptableFn * queueOrderFn * reclaimFn * jobOrderFn * namespaceOrderFn	为所有队列提供公平的资源共享。



Plugins

ID	Name	Arguments	Registered Functions	Description
4	extender	* extender.urlPrefix * extender.httpTimeout * extender.onSessionOpenVerb * extender.onSessionCloseVerb * extender.predicateVerb * extender.prioritizeVerb * extender.preemptableVerb * extender.reclaimableVerb * extender.queueOverusedVerb * extender.jobEnqueueableVerb * extender.ignorable	* predicateFn * batchNodeOrderFn * preemptableFn * reclaimableFn * jobEnqueueableFn * overusedFn	添加外部http server，用以执行自定义action。
5	gang	/	* jobValidFn * reclaimableFn * preemptableFn * jobOrderFn * JobReadyFn * jobPipelineFn * jobStarvingFn	为作业分配资源时，重点考虑作业的最低资源需求和pod最小运行数量，执行 “All or nothing”的调度策略。



Plugins

ID	Name	Arguments	Registered Functions	Description
6	nodeorder	* nodeaffinity.weight * podaffinity.weight * leastrequested.weight * balancedresource.weight * mostrequested.weight * tainttoleration.weight * imagelocality.weight	* nodeOrderFn * batchNodeOrderFn	以自定义的方式对所有节点进行排序。
7	numaaware	* weight	* predicateFn * batchNodeOrderFn	在将pod绑定到node节点时，重点考虑 CPU Numa 因素。
8	overcommit	* overcommit-factor	* jobEnqueueeableFn * jobEnqueuedFn	将可用资源设置为群集整体资源的指定倍数。

Plugins

ID	Name	Arguments	Registered Functions	Description
9	predicate	* predicate.GPUSharingEnable * predicate.CacheEnable * predicate.ProportionalEnable * predicate.resources * predicate.resources.nvidia.com/gpu.cpu * predicate.resources.nvidia.com/gpu.memory	* predicateFn	添加关于如何为pod调度过滤node的自定义函数。
10	priority	/	* taskOrderFn * jobOrderFn * preemptableFn * jobStarvingFn	定义调度作业的优先级。
11	proportion	/	* queueOrderFn * reclaimableFn * overusedFn * allocatableFn * jobEnqueueableFn	根据queue的配置，将集群的整个资源按比例划分到所有的queue。



Plugins

ID	Name	Arguments	Registered Functions	Description
12	reservation	/	* targetJobFn * reservedNodesFn	将节点按资源使用情况排序，并锁定部分资源，预留给目标作业。
13	sla	* sla-waiting-time	* jobOrderFn * jobEnqueueableFn * JobPipelinedFn	根据SLA设置对工作负载进行排序。
14	task-topology	/	* taskOrderFn * nodeOrderFn	根据给定策略，将不同角色的Pod绑定到节点上。
15	tdm	* tdm.revocable-zone.rz1 * tdm.revocable-zone.rz2 * tdm.evict.period	* predicateFn * nodeOrderFn * preemptableFn * victimTasksFn * jobOrderFn * jobPipelinedFn * jobStarvingFn	在不同的时间段内，允许部分节点承接K8s和其他集群的作业调度。



Join Volcano



Website: <https://volcano.sh/en/>



Github: <https://github.com/volcano-sh/volcano>



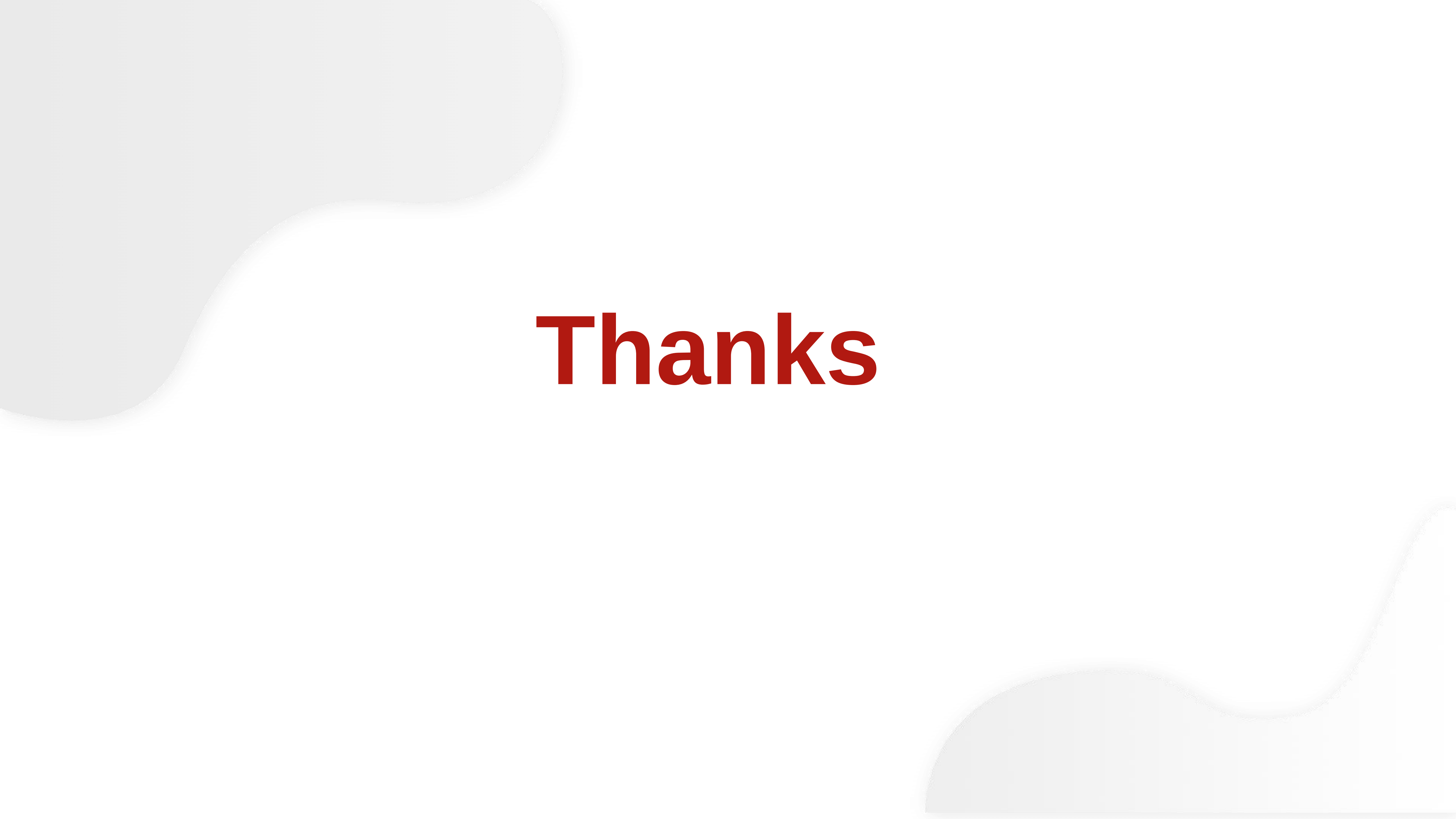
Slack Channel: <https://volcano-sh.slack.com/>



Follow Us



Join Group



Thanks